

CS 370 Module Two Assignment

To Begin: Use this **LastName_FirstName_Assignment2.ipynb** file to complete your assignment. Look for the section header **Start of Rubric Graded Items** for the placeholders for your assignment.

When Finished: When you have completed your work, download the project as an HTML file and submit the HTML file. Make sure that your file is named as specified in the rubric, and that you have addressed all of the rubric criteria in your response.

Hint: The code samples from the **Deep Learning with Keras** required readings pages 16-21 have been pre-loaded for you in the **Example_Code_Assignment2.ipynb** file. Some of the foundational starter code has also been added here as a starting point for your assignment as applicable.

Note: For compatability purposes, libraries have been updated from those used in the required readings to match to current versions; hence some of the package invocations may differ slightly from the book. The affected lines of code have comments added to the right as applicable, with the old code commented out above for reference.

GPU/CUDA/Memory Warnings/Errors: You may receive some errors referencing that GPUs will not be used, CUDA could not be found, or free system memory allocation errors. These and a few others, are standard errors that can be ignored here as they are environment based.

Example messages:

- Could not find cuda drivers on your machine, GPU will not be used.
- Please check linkage and avoid linking the same target more than once.
- E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
- Allocation of ##### exceeds 10% of free system memory

Required Baseline Starter Code

Installing Required Packages

This is to install necessary components to run the assignment

Hint: Ensure that you run all starter code blocks before running your code from the **Start of Rubric Graded Items** sections.

In [1]: `!pip install -r requirements.txt`

```
Requirement already satisfied: numpy==1.26 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from -r requirements.txt (line 1)) (1.26.0)
Requirement already satisfied: keras==3.10.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from -r requirements.txt (line 2)) (3.10.0)
Requirement already satisfied: absl-py in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (2.1.0)
Requirement already satisfied: rich in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (13.7.1)
Requirement already satisfied: namex in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (0.0.8)
Requirement already satisfied: h5py in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (3.11.0)
Requirement already satisfied: optree in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (0.12.1)
Requirement already satisfied: ml-dtypes in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (0.5.4)
Requirement already satisfied: packaging in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from keras==3.10.0->-r requirements.txt (line 2)) (24.1)
Requirement already satisfied: typing-extensions>=4.5.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from optree->keras==3.10.0->-r requirements.txt (line 2)) (4.12.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from rich->keras==3.10.0->-r requirements.txt (line 2)) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from rich->keras==3.10.0->-r requirements.txt (line 2)) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /home/codio/.pyenv/versions/3.11.9/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->rich->keras==3.10.0->-r requirements.txt (line 2)) (0.1.2)
```

[notice] A new release of pip is available: 24.1.2 -> 26.1.1

[notice] To update, run: `pip install --upgrade pip`

Loading the Required Libraries

Warnings: As a reminder, you may receive some errors here, these are standard errors that can be ignored as they are environment based.

Potentially helpful links:

- <https://keras.io/>
- <https://keras.io/api/datasets/>
- <https://keras.io/api/models/>
- <https://keras.io/api/layers/>
- <https://keras.io/api/optimizers/>
- https://keras.io/api/utils/python_utils/

```
In [2]: from __future__ import print_function
import numpy as np
from keras.datasets import mnist
from keras.models import Sequential
#from keras.layers.code import Dense, Activation # From the book, code replaced below
from keras.layers import Dense, Activation, Input # syntax change due to version bump
from keras.optimizers import SGD
#from keras.utils import np_utils # From the book, code replaced below
from keras.utils import to_categorical # syntax change due to version bump
```

```
-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

Generating a Numpy Random Seed For Reproducibility

Note: The best practice is to not reseed a BitGenerator, but rather to recreate a new one. While this method for the the seed generation is considered a legacy function, here we use it to as a way to ensure reproducible results for repeatble course submissions.

Potentially helpful links:

- https://keras.io/api/utils/python_utils/

```
In [3]: np.random.seed(1671) # for reproducibility
```

Network and Training Parameters

Warnings: As a reminder, you may receive some errors here, these are standard errors that can be ignored as they are environment based.

```
In [4]: NB_EPOCH = 20
        BATCH_SIZE = 128
        VERBOSE = 1
        NB_CLASSES = 10 # number of outputs = number of digits
        OPTIMIZER = SGD() # SGD optimizer, explained later in this chapter
        N_HIDDEN = 128
        VALIDATION_SPLIT = 0.2 # how much TRAIN is reserved for VALIDATION
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1779429443.660416      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.423034      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.426139      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.429797      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.432450      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.434836      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.639173      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.640922      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
I0000 00:00:1779429444.642539      386 cuda_executor.cc:1015] successful NUMA node re
ad from SysFS had negative value (-1), but there must be at least one NUMA node, so
returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Do
cumentation/ABI/testing/sysfs-bus-pci#L344-L355
```

Splitting Data Into Train and Test Sets

```
In [5]: # data: shuffled and split between train and test sets
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

Shaping

```
In [6]: # X_train is 60000 rows of 28x28 values --> reshaped in 60000 x 784
```

```
RESHAPED = 784

X_train = X_train.reshape(60000, RESHAPED)
X_test = X_test.reshape(10000, RESHAPED)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
```

```
In [7]: # normalize
X_train /= 255
X_test /= 255

print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')
```

60000 train samples
10000 test samples

```
In [8]: # convert class vectors to binary class matrices

# Y_train = np_utils.to_categorical(y_train, NB_CLASSES) # From the book, code repl
Y_train = to_categorical(y_train, NB_CLASSES) # Updated for current version

#Y_test = np_utils.to_categorical(y_test, NB_CLASSES) # From the book, code replace
Y_test = to_categorical(y_test, NB_CLASSES) # Updated for current version
```

Start of Rubric Graded Items

To Begin: Once you have reviewed the output of the blocks above, use the blocks below to complete your assignment portions.

Hint: As a reminder, the code samples from the **Deep Learning with Keras** required readings pages 16-21 have been pre-loaded for you in the **Example_Code_Assignment2.ipynb** file. Some of the foundational starter code has also been added here as a starting point for your assignment as applicable in the cells above.

Notebook Configuration and Naming Convention

Click on LastName_FirstName_Assignment2.ipynb in the filetree and rename it following that convention. For example, if your name is Jane Doe, you would rename the file Doe_Jane_Assignment2.ipynb for later submission.

Establishes Accuracy Scores with Two Hidden Layers

Follow the examples in the book to establish an accuracy rate for the training, validation, and test data sets with two hidden layers.

Note: The example model configurations provided below must be modified for your assignment submission. Some lines are standard commands.

Hint: Read through the MNIST handwritten digits example on pages 16-24 of **Deep Learning with Keras**. The code from pages 16-18 have been copied over to this Jupyter Notebook file for you as applicable in the cells above for the minimum required baseline code. The hidden layers examples are provided starting on page 22, ensure to keep example modifications from above in mind when building out the rest of your assignment code blocks. The code below can be copied and used to start building out your hidden layers task.

Book Example Code Snippet: A simple neural net in Keras

```
# 10 outputs
# final stage is softmax
model = Sequential()
# model.add(Dense(NB_CLASSES, input_shape=(RESHAPED,))) # From the
# book, code replaced below
model.add(Input(shape=(RESHAPED,))) # Updated for current version
model.add(Dense(NB_CLASSES)) # Updated for current version
model.add(Activation('softmax'))
model.summary()
model.compile(loss='categorical_crossentropy', optimizer=OPTIMIZER,
              metrics=['accuracy'])
history = model.fit(X_train, Y_train, batch_size=BATCH_SIZE,
                    epochs=NB_EPOCH, verbose=VERBOSE,
                    validation_split=VALIDATION_SPLIT)
score = model.evaluate(X_test, Y_test, verbose=VERBOSE)
```

```
print("Test Score:", score[0])  
print("Test Accuracy:", score[1])
```

Warnings: As a reminder, you may receive some errors in the model here, these are standard errors that can be ignored as they are environment based.

Potentially helpful links:

- <https://keras.io/api/models/sequential/>
- <https://keras.io/api/layers/activations/>
- <https://keras.io/api/models/sequential/#add-method>
- <https://keras.io/api/layers/>
- https://keras.io/api/layers/core_layers/
- https://keras.io/api/models/model_training_apis/#compile-method
- https://keras.io/api/models/model_training_apis/#fit-method
- https://keras.io/api/models/model_training_apis/#evaluate-method

In []:

Establishes Accuracy Scores by Varying Parameters

Pick one parameter and run two or three different experiments modifying the parameter values to establish accuracy scores with different parameter values. Make sure the experiments result in significant changes in accuracy rates. Be sure to place each experiment in a different code block so that your instructor can view all of your changes.

Note: You may have to do some research beyond the information provided in the book to implement these changes. Remember to copy your model definition, compilation, training, and evaluation blocks to test your experiment.

Hint: Sample parameters that you can overwrite here were used in the **Network and Training Parameters** section above. This is not all inclusive and just provided as a baseline. See the additional example parameters starting on page 25 in the **Deep Learning with Keras** book.

Important: When codifying your experiments, ensure that you include the replacement variable in the cell below, followed by the hidden layers code you developed in the rubric item above.

Book Example Code Snippet: Copied from cell above for quick reference to variable naming

```
NB_EPOCH = 20
BATCH_SIZE = 128
VERBOSE = 1
NB_CLASSES = 10 # number of outputs = number of digits
OPTIMIZER = SGD() # SGD optimizer, explained later in this chapter
N_HIDDEN = 128
VALIDATION_SPLIT = 0.2 # how much TRAIN is reserved for VALIDATION
```

Potentially helpful links:

- https://keras.io/api/models/model_training_apis/#fit-method

Experiment 1

Hint: When modifying parameters that will impact the optimization of the solution, ensure that you add the **OPTIMIZER = SGD()** line before your code with the hidden layers from above.

```
In [9]: from __future__ import print_function
import numpy as np
from keras.datasets import mnist
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
from keras.models import Sequential
#from keras.layers import Dense, Activation # From the book, code replaced below
from keras.layers import Dense, Activation, Input # syntax change due to version bump
from keras.optimizers import SGD
#from keras.utils import np_utils # From the book, code replaced below
from keras.utils import to_categorical # syntax change due to version bump

np.random.seed(1671) # for reproducibility

#NB_EPOCH = 200 # From the book, code replaced below
NB_EPOCH = 20 # syntax change to speed up quick look testing
BATCH_SIZE = 128
VERBOSE = 1
NB_CLASSES = 10 # number of outputs = number of digits
OPTIMIZER = SGD() # SGD optimizer, explained later in this chapter
N_HIDDEN = 100
VALIDATION_SPLIT = 0.2 # how much TRAIN is reserved for VALIDATION

# data: shuffled and split between train and test sets
(X_train, y_train), (X_test, y_test) = mnist.load_data()

RESHAPED = 784

X_train = X_train.reshape(60000, RESHAPED)
```



```

X_test = X_test.reshape(10000, RESHAPED)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

X_train /= 255
X_test /= 255

print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')

Y_train = to_categorical(y_train, NB_CLASSES) # Updated for current version

#Y_test = np_utils.to_categorical(y_test, NB_CLASSES) # From the book, code replace
Y_test = to_categorical(y_test, NB_CLASSES) # Updated for current version

model = Sequential()
#model.add(Dense(NB_CLASSES, input_shape=(RESHAPED,))) # From the book, code replac
model.add(Input(shape=(RESHAPED,))) # Updated for current version
model.add(Dense(NB_CLASSES)) # Updated for current version
model.add(Activation('softmax'))
model.summary()

model.compile(loss='categorical_crossentropy', optimizer=OPTIMIZER, metrics=['accur

history = model.fit(X_train, Y_train, batch_size=BATCH_SIZE, epochs=NB_EPOCH, verbo

score = model.evaluate(X_test, Y_test, verbose=VERBOSE)

print("Test Score:", score[0])
print("Test Accuracy:", score[1])

```

60000 train samples

10000 test samples

Model: "sequential"

Layer (type)	Output Shape	Par
dense (Dense)	(None, 10)	7
activation (Activation)	(None, 10)	



Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1779429447.362941 466 service.cc:146] XLA service 0x781de0004fb0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices: I0000 00:00:1779429447.362969 466 service.cc:154] StreamExecutor device (0): Tesla T4, Compute Capability 7.5

91/375 ————— 0s 2ms/step - accuracy: 0.2439 - loss: 2.2277

```
I0000 00:00:1779429447.938568    466 device_compiler.h:188] Compiled cluster using  
XLA! This line is logged at most once for the lifetime of the process.
```

375/375  2s 4ms/step - accuracy: 0.4874 - loss: 1.7948 - val_accuracy: 0.8161 - val_loss: 0.9053
Epoch 2/20

375/375  1s 2ms/step - accuracy: 0.8129 - loss: 0.8594 - val_accuracy: 0.8506 - val_loss: 0.6627
Epoch 3/20

375/375  1s 2ms/step - accuracy: 0.8433 - loss: 0.6728 - val_accuracy: 0.8678 - val_loss: 0.5649
Epoch 4/20

375/375  1s 2ms/step - accuracy: 0.8573 - loss: 0.5882 - val_accuracy: 0.8755 - val_loss: 0.5113
Epoch 5/20

375/375  1s 2ms/step - accuracy: 0.8632 - loss: 0.5406 - val_accuracy: 0.8807 - val_loss: 0.4766
Epoch 6/20

375/375  1s 2ms/step - accuracy: 0.8691 - loss: 0.5088 - val_accuracy: 0.8842 - val_loss: 0.4519
Epoch 7/20

375/375  1s 2ms/step - accuracy: 0.8729 - loss: 0.4832 - val_accuracy: 0.8872 - val_loss: 0.4336
Epoch 8/20

375/375  1s 3ms/step - accuracy: 0.8768 - loss: 0.4663 - val_accuracy: 0.8906 - val_loss: 0.4191
Epoch 9/20

375/375  1s 2ms/step - accuracy: 0.8807 - loss: 0.4512 - val_accuracy: 0.8925 - val_loss: 0.4074
Epoch 10/20

375/375  1s 2ms/step - accuracy: 0.8857 - loss: 0.4339 - val_accuracy: 0.8939 - val_loss: 0.3979
Epoch 11/20

375/375  1s 2ms/step - accuracy: 0.8865 - loss: 0.4233 - val_accuracy: 0.8954 - val_loss: 0.3895
Epoch 12/20

375/375  1s 2ms/step - accuracy: 0.8873 - loss: 0.4166 - val_accuracy: 0.8976 - val_loss: 0.3824
Epoch 13/20

375/375  1s 2ms/step - accuracy: 0.8904 - loss: 0.4050 - val_accuracy: 0.8985 - val_loss: 0.3763
Epoch 14/20

375/375  1s 2ms/step - accuracy: 0.8919 - loss: 0.3987 - val_accuracy: 0.9003 - val_loss: 0.3707
Epoch 15/20

375/375  1s 2ms/step - accuracy: 0.8912 - loss: 0.3979 - val_accuracy: 0.9015 - val_loss: 0.3660
Epoch 16/20

375/375  1s 2ms/step - accuracy: 0.8925 - loss: 0.3934 - val_accuracy: 0.9022 - val_loss: 0.3618
Epoch 17/20

375/375  1s 2ms/step - accuracy: 0.8928 - loss: 0.3870 - val_accuracy: 0.9040 - val_loss: 0.3578
Epoch 18/20

375/375  1s 2ms/step - accuracy: 0.8939 - loss: 0.3838 - val_accuracy: 0.9053 - val_loss: 0.3542
Epoch 19/20

375/375  1s 2ms/step - accuracy: 0.8969 - loss: 0.3749 - val_accuracy: 0.9055 - val_loss: 0.3510

Epoch 20/20

375/375 ————— 1s 2ms/step - accuracy: 0.8972 - loss: 0.3727 - val_acc
uracy: 0.9062 - val_loss: 0.3481

313/313 ————— 1s 3ms/step - accuracy: 0.8906 - loss: 0.3991

Test Score: 0.34910547733306885

Test Accuracy: 0.9052000045776367

Experiment 2

Hint: When modifying parameters that will impact the optimization of the solution, ensure that you add the **OPTIMIZER = SGD()** line before your code with the hidden layers from above.

```
In [11]: from __future__ import print_function
import numpy as np
from keras.datasets import mnist
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
from keras.models import Sequential
#from keras.layers import Dense, Activation # From the book, code replaced below
from keras.layers import Dense, Activation, Input # syntax change due to version bump
from keras.optimizers import SGD
#from keras.utils import np_utils # From the book, code replaced below
from keras.utils import to_categorical # syntax change due to version bump

np.random.seed(1671) # for reproducibility

#NB_EPOCH = 200 # From the book, code replaced below
NB_EPOCH = 20 # syntax change to speed up quick look testing
BATCH_SIZE = 128
VERBOSE = 1
NB_CLASSES = 10 # number of outputs = number of digits
OPTIMIZER = SGD() # SGD optimizer, explained later in this chapter
N_HIDDEN = 100
VALIDATION_SPLIT = 0.3 # how much TRAIN is reserved for VALIDATION

# data: shuffled and split between train and test sets
(X_train, y_train), (X_test, y_test) = mnist.load_data()

RESHAPED = 784

X_train = X_train.reshape(60000, RESHAPED)
X_test = X_test.reshape(10000, RESHAPED)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

X_train /= 255
X_test /= 255

print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')

Y_train = to_categorical(y_train, NB_CLASSES) # Updated for current version
```

```

#Y_test = np_utils.to_categorical(y_test, NB_CLASSES) # From the book, code replace
Y_test = to_categorical(y_test, NB_CLASSES) # Updated for current version

model = Sequential()
#model.add(Dense(NB_CLASSES, input_shape=(RESHAPED,))) # From the book, code replac
model.add(Input(shape=(RESHAPED,))) # Updated for current version
model.add(Dense(NB_CLASSES)) # Updated for current version
model.add(Activation('softmax'))
model.summary()

model.compile(loss='categorical_crossentropy', optimizer=OPTIMIZER, metrics=['accu

history = model.fit(X_train, Y_train, batch_size=BATCH_SIZE, epochs=NB_EPOCH, verbo

score = model.evaluate(X_test, Y_test, verbose=VERBOSE)

print("Test Score:", score[0])
print("Test Accuracy:", score[1])

```

60000 train samples

10000 test samples

Model: "sequential_1"

Layer (type)	Output Shape	Par
dense_1 (Dense)	(None, 20)	15
activation_1 (Activation)	(None, 20)	



Total params: 15,700 (61.33 KB)

Trainable params: 15,700 (61.33 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20
329/329 ————— 3s 6ms/step - accuracy: 0.4475 - loss: 2.0685 - val_acc
uracy: 0.8122 - val_loss: 0.9940
Epoch 2/20
329/329 ————— 1s 3ms/step - accuracy: 0.8116 - loss: 0.9237 - val_acc
uracy: 0.8486 - val_loss: 0.7180
Epoch 3/20
329/329 ————— 1s 3ms/step - accuracy: 0.8453 - loss: 0.6989 - val_acc
uracy: 0.8623 - val_loss: 0.6091
Epoch 4/20
329/329 ————— 1s 3ms/step - accuracy: 0.8555 - loss: 0.6117 - val_acc
uracy: 0.8694 - val_loss: 0.5496
Epoch 5/20
329/329 ————— 1s 3ms/step - accuracy: 0.8651 - loss: 0.5570 - val_acc
uracy: 0.8746 - val_loss: 0.5113
Epoch 6/20
329/329 ————— 1s 3ms/step - accuracy: 0.8717 - loss: 0.5202 - val_acc
uracy: 0.8791 - val_loss: 0.4843
Epoch 7/20
329/329 ————— 1s 2ms/step - accuracy: 0.8724 - loss: 0.5015 - val_acc
uracy: 0.8820 - val_loss: 0.4639
Epoch 8/20
329/329 ————— 1s 3ms/step - accuracy: 0.8805 - loss: 0.4736 - val_acc
uracy: 0.8851 - val_loss: 0.4482
Epoch 9/20
329/329 ————— 1s 3ms/step - accuracy: 0.8822 - loss: 0.4611 - val_acc
uracy: 0.8871 - val_loss: 0.4353
Epoch 10/20
329/329 ————— 1s 3ms/step - accuracy: 0.8840 - loss: 0.4448 - val_acc
uracy: 0.8894 - val_loss: 0.4248
Epoch 11/20
329/329 ————— 1s 3ms/step - accuracy: 0.8883 - loss: 0.4282 - val_acc
uracy: 0.8910 - val_loss: 0.4159
Epoch 12/20
329/329 ————— 1s 3ms/step - accuracy: 0.8889 - loss: 0.4297 - val_acc
uracy: 0.8929 - val_loss: 0.4082
Epoch 13/20
329/329 ————— 1s 3ms/step - accuracy: 0.8905 - loss: 0.4133 - val_acc
uracy: 0.8940 - val_loss: 0.4012
Epoch 14/20
329/329 ————— 1s 3ms/step - accuracy: 0.8918 - loss: 0.4043 - val_acc
uracy: 0.8946 - val_loss: 0.3953
Epoch 15/20
329/329 ————— 1s 3ms/step - accuracy: 0.8946 - loss: 0.3947 - val_acc
uracy: 0.8959 - val_loss: 0.3902
Epoch 16/20
329/329 ————— 1s 3ms/step - accuracy: 0.8948 - loss: 0.3990 - val_acc
uracy: 0.8967 - val_loss: 0.3853
Epoch 17/20
329/329 ————— 1s 3ms/step - accuracy: 0.8961 - loss: 0.3892 - val_acc
uracy: 0.8981 - val_loss: 0.3810
Epoch 18/20
329/329 ————— 1s 3ms/step - accuracy: 0.8972 - loss: 0.3850 - val_acc
uracy: 0.8978 - val_loss: 0.3770
Epoch 19/20
329/329 ————— 1s 3ms/step - accuracy: 0.8983 - loss: 0.3799 - val_acc

uracy: 0.8997 - val_loss: 0.3736

Epoch 20/20

329/329 ————— 1s 3ms/step - accuracy: 0.8984 - loss: 0.3746 - val_acc

uracy: 0.9003 - val_loss: 0.3706

313/313 ————— 1s 2ms/step - accuracy: 0.8873 - loss: 0.4072

Test Score: 0.3547130525112152

Test Accuracy: 0.9024999737739563

Experiment 3 (Optional)

Hint: When modifying parameters that will impact the optimization of the solution, ensure that you add the **OPTIMIZER = SGD()** line before your code with the hidden layers from above.

```
In [1]: from __future__ import print_function
import numpy as np
from keras.datasets import mnist
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
from keras.models import Sequential
#from keras.layers import Dense, Activation # From the book, code replaced below
from keras.layers import Dense, Activation, Input # syntax change due to version bump
from keras.optimizers import SGD
#from keras.utils import np_utils # From the book, code replaced below
from keras.utils import to_categorical # syntax change due to version bump

np.random.seed(1671) # for reproducibility

#NB_EPOCH = 200 # From the book, code replaced below
NB_EPOCH = 20 # syntax change to speed up quick look testing
BATCH_SIZE = 128
VERBOSE = 1
NB_CLASSES = 10 # number of outputs = number of digits
OPTIMIZER = SGD() # SGD optimizer, explained later in this chapter
N_HIDDEN = 50
VALIDATION_SPLIT = 0.3 # how much TRAIN is reserved for VALIDATION

# data: shuffled and split between train and test sets
(X_train, y_train), (X_test, y_test) = mnist.load_data()

RESHAPED = 784

X_train = X_train.reshape(60000, RESHAPED)
X_test = X_test.reshape(10000, RESHAPED)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

X_train /= 255
X_test /= 255

print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')
```

```

Y_train = to_categorical(y_train, NB_CLASSES) # Updated for current version

#Y_test = np_utils.to_categorical(y_test, NB_CLASSES) # From the book, code replace
Y_test = to_categorical(y_test, NB_CLASSES) # Updated for current version

model = Sequential()
#model.add(Dense(NB_CLASSES, input_shape=(RESHAPED,))) # From the book, code replace
model.add(Input(shape=(RESHAPED,))) # Updated for current version
model.add(Dense(NB_CLASSES)) # Updated for current version
model.add(Activation('softmax'))
model.summary()

model.compile(loss='categorical_crossentropy', optimizer=OPTIMIZER, metrics=['accuracy'])

history = model.fit(X_train, Y_train, batch_size=BATCH_SIZE, epochs=NB_EPOCH, verbose=1)

score = model.evaluate(X_test, Y_test, verbose=VERBOSE)

print("Test Score:", score[0])
print("Test Accuracy:", score[1])

```

```

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

```

```

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

```

```

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

```

```

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

```

```

-----
AttributeError                                Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'

```

60000 train samples

10000 test samples

Model: "sequential"

Layer (type)	Output Shape	Par
dense (Dense)	(None , 10)	7
activation (Activation)	(None , 10)	



Total params: [7,850](#) (30.66 KB)

Trainable params: [7,850](#) (30.66 KB)

Non-trainable params: [0](#) (0.00 B)

Epoch 1/20
329/329  3s 7ms/step - accuracy: 0.5133 - loss: 1.7538 - val_accuracy: 0.8149 - val_loss: 0.9492

Epoch 2/20
329/329  2s 6ms/step - accuracy: 0.8159 - loss: 0.8900 - val_accuracy: 0.8477 - val_loss: 0.7032

Epoch 3/20
329/329  2s 6ms/step - accuracy: 0.8450 - loss: 0.6946 - val_accuracy: 0.8604 - val_loss: 0.6020

Epoch 4/20
329/329  2s 5ms/step - accuracy: 0.8589 - loss: 0.6003 - val_accuracy: 0.8692 - val_loss: 0.5455

Epoch 5/20
329/329  2s 5ms/step - accuracy: 0.8647 - loss: 0.5514 - val_accuracy: 0.8748 - val_loss: 0.5085

Epoch 6/20
329/329  1s 4ms/step - accuracy: 0.8677 - loss: 0.5234 - val_accuracy: 0.8783 - val_loss: 0.4822

Epoch 7/20
329/329  2s 5ms/step - accuracy: 0.8756 - loss: 0.4882 - val_accuracy: 0.8814 - val_loss: 0.4625

Epoch 8/20
329/329  2s 5ms/step - accuracy: 0.8798 - loss: 0.4644 - val_accuracy: 0.8841 - val_loss: 0.4471

Epoch 9/20
329/329  2s 5ms/step - accuracy: 0.8823 - loss: 0.4524 - val_accuracy: 0.8862 - val_loss: 0.4347

Epoch 10/20
329/329  2s 5ms/step - accuracy: 0.8845 - loss: 0.4388 - val_accuracy: 0.8884 - val_loss: 0.4242

Epoch 11/20
329/329  2s 5ms/step - accuracy: 0.8865 - loss: 0.4290 - val_accuracy: 0.8894 - val_loss: 0.4153

Epoch 12/20
329/329  1s 4ms/step - accuracy: 0.8870 - loss: 0.4254 - val_accuracy: 0.8916 - val_loss: 0.4077

Epoch 13/20
329/329  2s 5ms/step - accuracy: 0.8883 - loss: 0.4171 - val_accuracy: 0.8928 - val_loss: 0.4008

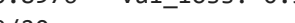
Epoch 14/20
329/329  2s 5ms/step - accuracy: 0.8909 - loss: 0.4079 - val_accuracy: 0.8940 - val_loss: 0.3949

Epoch 15/20
329/329  2s 5ms/step - accuracy: 0.8934 - loss: 0.3963 - val_accuracy: 0.8946 - val_loss: 0.3897

Epoch 16/20
329/329  2s 5ms/step - accuracy: 0.8923 - loss: 0.3958 - val_accuracy: 0.8958 - val_loss: 0.3852

Epoch 17/20
329/329  2s 5ms/step - accuracy: 0.8958 - loss: 0.3867 - val_accuracy: 0.8968 - val_loss: 0.3809

Epoch 18/20
329/329  2s 5ms/step - accuracy: 0.8972 - loss: 0.3794 - val_accuracy: 0.8976 - val_loss: 0.3774

Epoch 19/20
329/329  2s 5ms/step - accuracy: 0.8947 - loss: 0.3847 - val_accuracy: 0.8947 - val_loss: 0.3847

uracy: 0.8989 - val_loss: 0.3735

Epoch 20/20

329/329  2s 5ms/step - accuracy: 0.8981 - loss: 0.3778 - val_acc

uracy: 0.8995 - val_loss: 0.3707

313/313  1s 3ms/step - accuracy: 0.8891 - loss: 0.4051

Test Score: 0.3544345796108246

Test Accuracy: 0.9046000242233276

Explains Changes in Accuracy Rates

Create a Markdown cell in your Jupyter Notebook after your code and its outputs. In this cell, explain the changes in accuracy rates by comparing and contrasting your results from Steps 3 and 4. What happens to the accuracy rates for the training, validation, and test data sets as you change the parameters. Why?

Hint: To use the markdown block below, double click in the **Type Markdown and LaTeX: α^2** block below, to turn it back to html, Run the cell.

Potentially helpful links:

- [https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Working%20With%20Markdown%](https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Working%20With%20Markdown%20Cells.html)



the base model without any changes we see the accuracy 90.5%. However when I changed the NB_classes value from the default 10 to 20 we see the accuracy actually decrease to 90.24% this makes increasing the output nodes can increase computational power needed and introduce more areas where the model can fail. in the last example halved the Hidden nodes this also decreased my accuracy however not by much. This makes senses as I am allocating less Neurons to my machine causing it to work with less.